

Performance

Q-KET

2026.09.04

CONTENTS

01 프로젝트 개요

주제 선정 배경 · 개발 환경 · 진행 일정 · 기대 효과

02 수행 절차 및 방법

사전 기획 단계 · 수행 단계 · 완료 단계

03 수행 경과

시스템 아키텍처 · 인프라 · 보안 · 모니터링 · 배포 결과

04 부하 테스트 및 성능 개선

테스트 설계 · 병목 분석 · 개선 방향 · 용량 산정

05 트러블슈팅 및 피드백

장애 사례와 해결 · 멘토 피드백 및 개선 내역

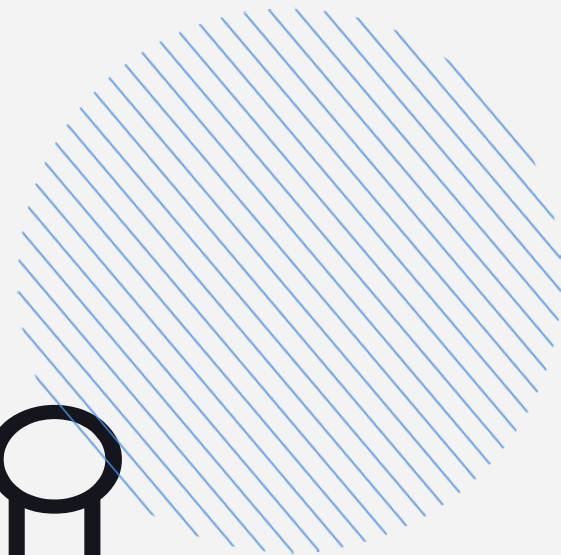
06 시연 및 자체 평가

시연 영상 · 완성도 평가 · 추후 개선점 · 느낀 점



Chapter 1

프로젝트 개요



- 주제 선정 배경과 기획 의도
- 3차 개선점과 목표
- 3차 신규 기능
- 개발 환경 및 사용 도구
- 팀 구성 및 역할
- 진행 일정

공연 예매 오픈은 짧은 순간에 트래픽이 집중되는 대표적인 워크로드입니다.

01

동시 접속 폭주

오픈 시각에 수천 명이 한꺼번에 접속하며 서버가 순간적으로 과부하 상태가 됩니다.

02

무한 재시도 및 트래픽 증폭

명확한 대기 체계가 없어 사용자가 지속적으로 새로고침을 시도하며 트래픽 폭증이 가속화됩니다.

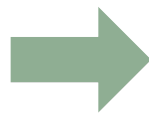
03

중복 예매 위험

같은 좌석에 여러 요청이 동시에 도달하면 한 좌석이 두 번 판매될 수 있습니다.

2차 프로젝트 - 결과물

- 대기열 · 동시성 제어 시스템을 설계하고 구현
- EKS 기반으로 dev · prod 환경 분리 배포
- GitHub Actions로 CI/CD 파이프라인 구축



3차 프로젝트 - 목표

- 실제 한계까지 부하를 걸어 병목현상을 측정
- 노드 · 파드 AutoScaling과 GitOps 고도화
- 측정 결과에 근거한 용량 산정과 개선안 도출

3차 프로젝트 목표

10,000명

목표 동시 접속

4,000명

실측 검증 규모

400명

대기열 동시 입장 제한

0건

중복 예매 허용 기준

2차에서 겪은 CI/CD 불안정 · 콘솔 기반 인프라 · 부하 대응 미흡 - 이 세 가지를 개선하기로 했습니다.

1

외부 API 사용

Toss Payments · 간편로그인

- 결제 위젯 연동 및 승인 처리
- 결제 완료 시점에 좌석 확정
- 환불 시 좌석 자동 반납
- 카카오·네이버·구글 간편로그인
- 회원 정보 자동 매핑 처리

2

감상평

AI 스포일러 판별

- 예매한 회차당 한번 작성 가능
- OpenAI로 스포일러 자동 판별
- 스포일러 댓글 클릭 후 노출

3

알림

SQS · Lambda · SES

- 회원가입 시 이메일 인증 코드 발송
- 미인증 계정 일정 시간 후 자동 만료
- 예매 확정/취소 시 메일 발송
- 오픈 30분 전 알림 신청
- 비동기 처리로 응답 지연 차단

4

보안 강화

인증 · 접근 제어

- 인증 미들웨어로 페이지 접근 제어
- 대기열 우회 취약점 차단
- 역할 기반 메뉴 접근 제어

파란 बै지 3차에서 새로 도입한 기술

클라이언트

Next.js 14

TypeScript

npm

애플리케이션

JAVA 21

Spring Boot 3.5

Gradle

MyBatis

데이터 · 동시성

MySQL 8.0 (RDS)

Redis 7 (ElastiCache)

Redisson

인프라

AWS EKS

Terraform

Karpenter

KEDA

CI / CD

Github Actions

Amazon ECR

ArgoCD (GitOps)

모니터링 · 검증

Prometheus

Grafana

k6

Loki

Grafana Faro

외부 연동

Toss Payments

OpenAI

SES · SQS · Lambda



이채영
팀장



나윤준
부팀장



강진호
팀원

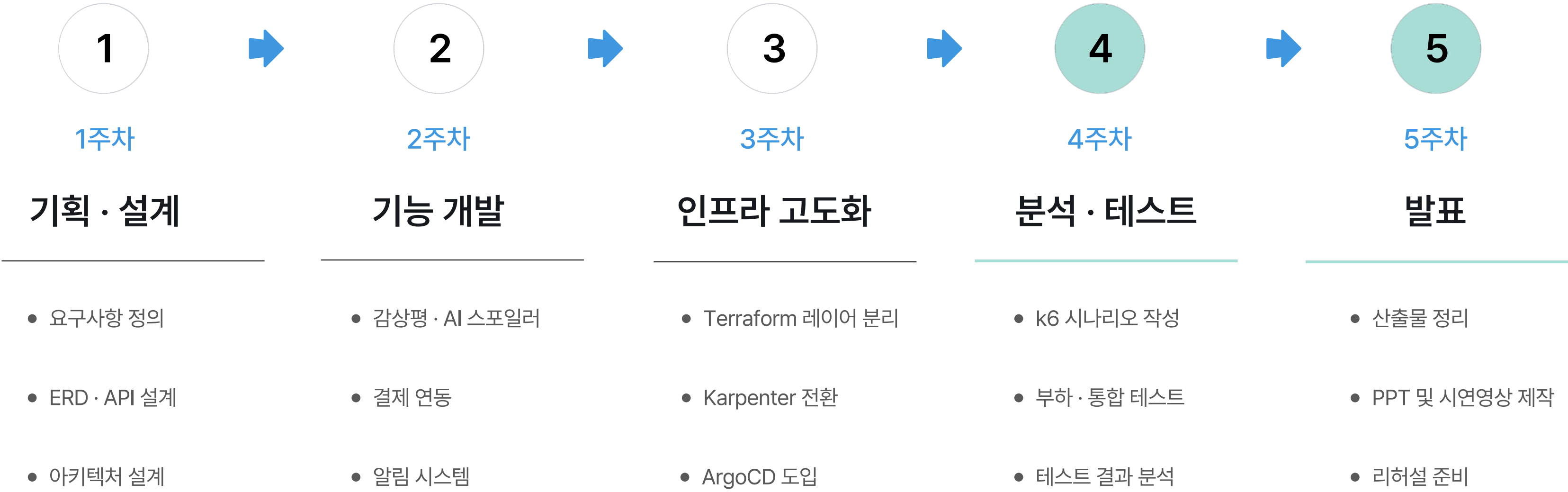


문준혁
팀원



정우진
팀원

네트워크 VPC · Subnet	●	●	●	●	●
스토리지 · DB RDS · S3 · Redis	●	●	●	●	●
인프라 EKS · ECR · ELB	●	●	●	●	●
CI / CD GitHub Actions · ArgoCD(GitOps)	●	●	●	●	●
애플리케이션 Next.js · Spring Boot API	●	●	●	●	●
모니터링 모니터링 · 부하테스트 및 서버 개선	●	●	●	●	●

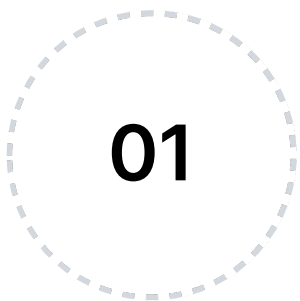




Chapter 2

수행절차 및 방법

- 수행 절차 개요와 협업 방식
- 단계별 수행 내용
- 기술 선택과 근거



사전 기획

1주차

요구사항 정의 · 설계
역할 분담

수행

2 ~ 3주차

기능 개발 · 인프라 고도화
통합 · 부하 테스트

검증

4 ~ 5주차

부하 테스트 · 병목 분석
문서화 · 회고

협업 방식

브랜치 전략

Git Flow

feature → dev → release → main 순으로 병합
배포는 main(prod) 기준

코드 리뷰

Pull Request 기반 리뷰 후 병합,
빌드 검사 자동 실행

문서화

장애 사례와 의사결정을 기록해 재발 방지
요구사항 정의서 · API 명세서 최신화

사전 기획

1주차

- ERD · API 설계 후 팀 리뷰
- 성능 목표 **동시 접속 10,000명** 확정
- 예매 좌석 사전 생성 구조 설계
- EKS · Redis Sorted Set 채택

수행

2 ~ 3주차

- 결제 · 감상평 · 알림 신규 기능 구현
- Terraform 레이어 분리 · Karpenter 전환
- release · prod 배포 검증
- 완성된 기능으로 **부하 시나리오 확보**
- 대기열 우회 취약점 차단

검증

4 ~ 5주차

- **병목 원인** 특정 및 개선안 도출
- 개선 적용 후 재측정
- 규모별 필요 자원 산정
- 문서화 및 발표 준비

기능을 먼저 완성해 실제 사용자 흐름을 확보한 뒤, 그 시나리오로 인프라의 한계를 측정했습니다.

대기열 구현

Redis Sorted Set ↔ DB 테이블

1만 명이 대기해도 순번 조회는 13번의 비교로 끝납니다. 세션 저장소로 이미 쓰는 Redis를 그대로 활용했습니다.

동시성 제어

Redisson 분산 락 ↔ DB 비관적 락

Pod가 여러 대로 늘어나면 DB 락만으로는 부족합니다. 락 경합 부하를 DB가 아닌 Redis가 받도록 분리했습니다.

알림 메시징

SQS ↔ SNS

SNS는 여러 구독자에게 같은 메시지를 뿌립니다. 예매 메일은 사용자마다 다른 내용을 한 건씩 보내고 실패 시 재처리해야 해서 작업 큐를 골랐습니다.

노드 스케일링

Karpenter ↔ Cluster Autoscaler

노드 그룹을 미리 정의하지 않고 요구 자원에 맞는 인스턴스를 직접 선택합니다. 버스트 상황에서 반응이 빠릅니다.

파드 스케일링

KEDA ↔ HPA

HPA는 CPU · 메모리로만 스케일합니다. 지금은 CPU 기준이지만 대기열 길이를 트리거로 추가할 수 있는 구조를 확보했습니다.

배포 방식

멘토 피드백 반영

수동 Sync + 메일 ↔ auto-sync

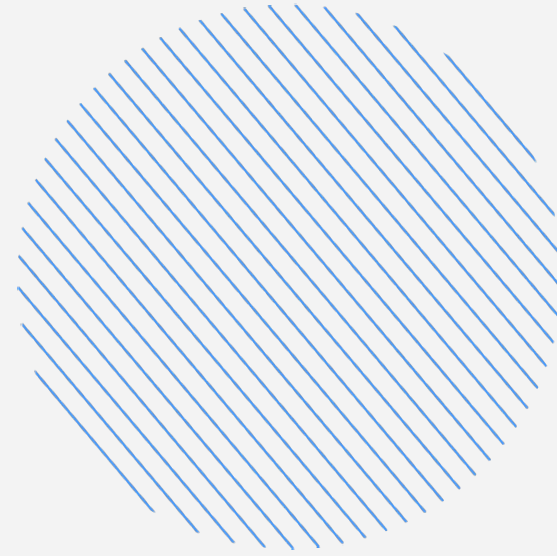
OutOfSync를 감지하면 메일로 알리고 배포 시점은 팀이 결정합니다. 기본 트리거에 감지 기능이 없어 직접 정의했습니다.

위 세 가지는 애플리케이션 설계, 아래 세 가지는 인프라 · 운영 결정입니다. 여섯 가지 모두 성능이 아니라 **운영 가능성**을 기준으로 골랐습니다.

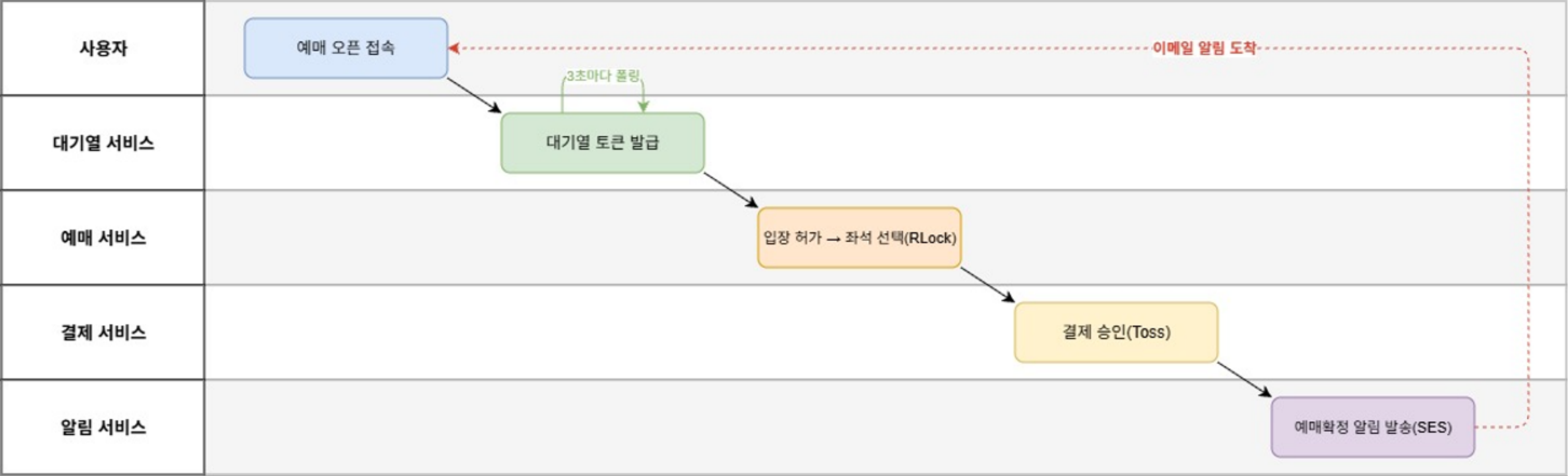


Chapter 3

수행 경과



- 서비스 동작 흐름
- AWS 인프라 구성과 네트워크
- 쿠버네티스 클러스터 구성
- 오토스케일링 동작 방식
- CI / CD 및 인프라 배포 전략
- ArgoCD
- 보안 구성과 관측 체계



대기열 접근

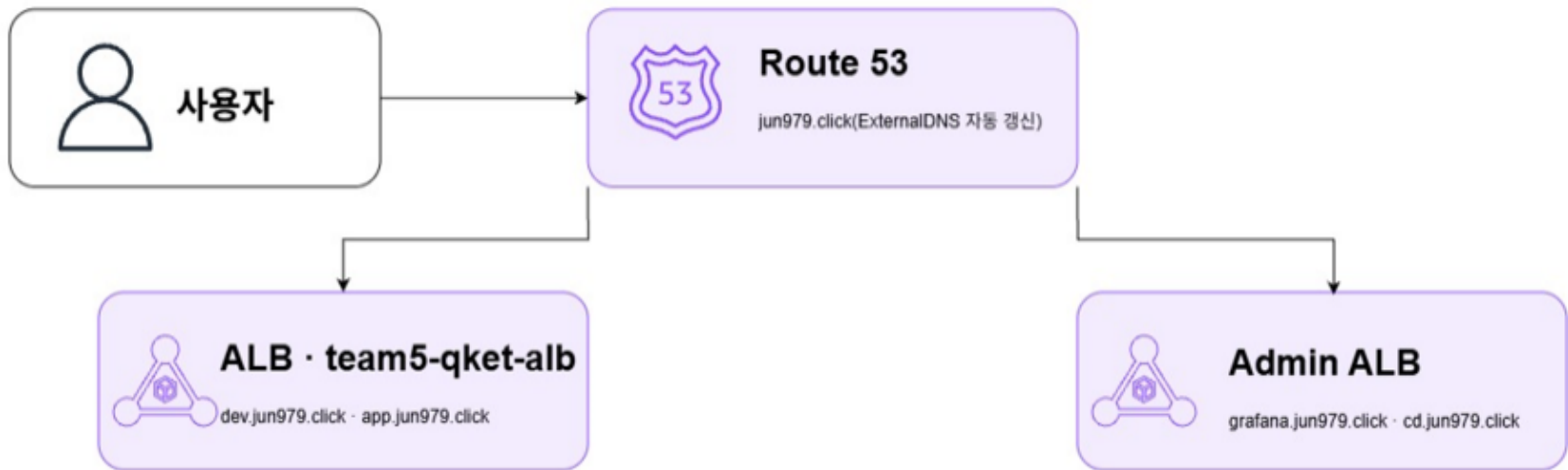
- 대기열 토큰 없이는 좌석 페이지에 접근 불가
- 3초마다 순번을 풀링

중복 차단

- 분산 락으로 1차 차단
- 조건부 갱신으로 최종 확정

응답 분리

- 알림은 SQS로 넘겨 비동기 처리
- 메일 발송이 예약 응답을 붙잡지 않음



네트워크 설계

2개 가용영역

한 AZ가 죽어도 서비스가 유지되도록
노드와 데이터베이스를 양쪽에 배치

3중 서브넷

Public(NAT) · Private(워커로드) · Private(데이터)로
분리해 DB를 외부에서 직접 접근 불가능하게 구성

SSM Bastion

SSH 포트를 열지 않고 Session Manager로만 내부 접근

함께 사용한 AWS 서비스

ECR 컨테이너 이미지

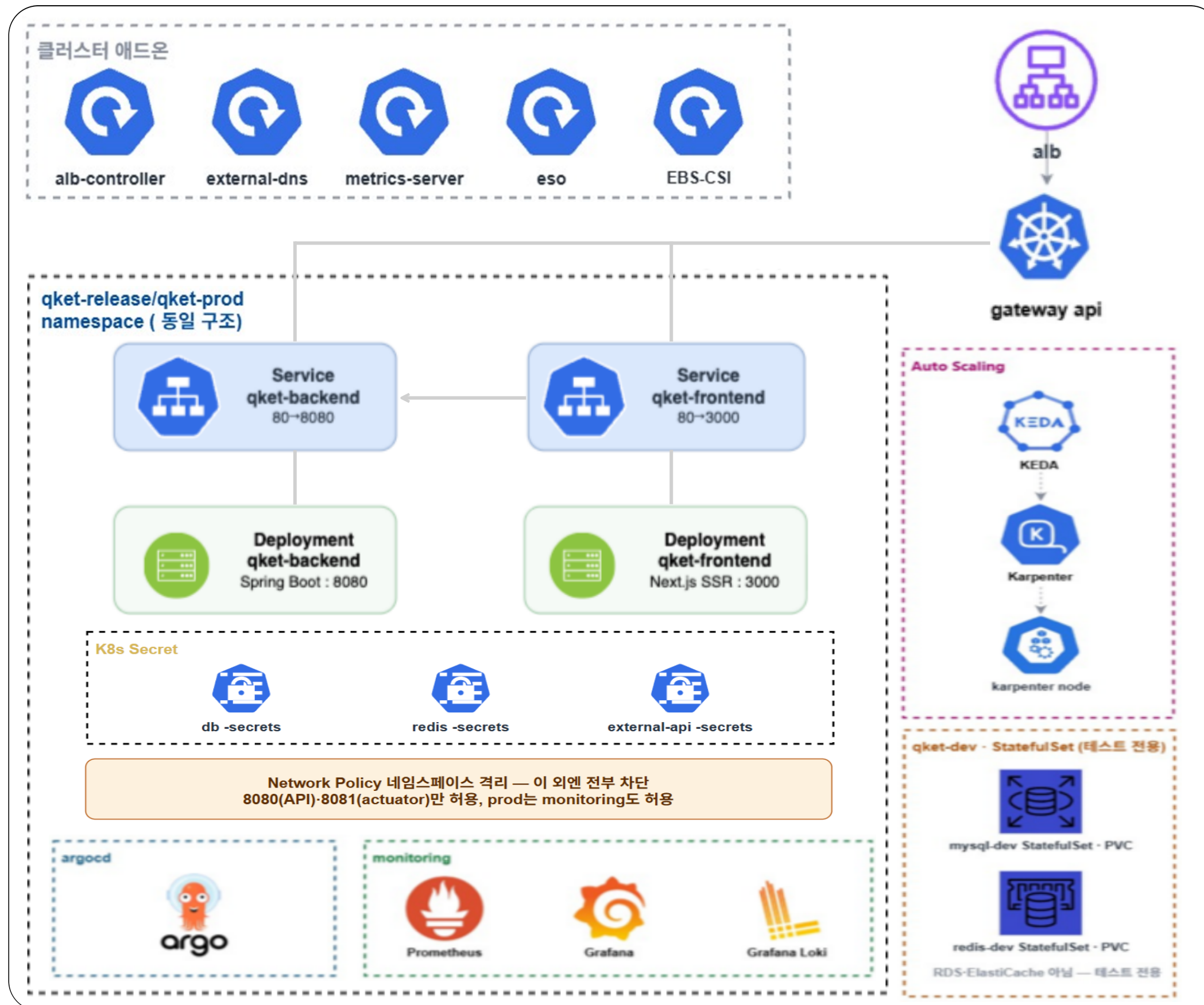
S3 · CloudFront 포스터

Secrets Manager 자격증명

SQS · Lambda · SES 알림

CloudWatch RDS · Redis 지표

AMP Prometheus



1

클러스터 애드온

ALB Controller가 Gateway 설정을 읽어 로드밸런서를 만들고, ExternalDNS가 Route 53 레코드를 자동으로 갱신합니다.

2

자격증명은 코드에 없습니다.

External Secrets Operator가 AWS Secrets Manager의 값을 읽어 K8s Secret으로 만듭니다. Git에는 비밀번호가 한 줄도 들어가지 않습니다.

3

NetworkPolicy로 격리

백엔드는 같은 네임스페이스에서 오는 8080 · 8081 요청만 받습니다. 른 네임스페이스나 다른 외부에서 직접 파드에 닿을 수 없습니다.

4

오토스케일링

KEDA가 파드를 늘리고, 었을 자리가 없으면 Karpenter가 노드를 만듭니다.

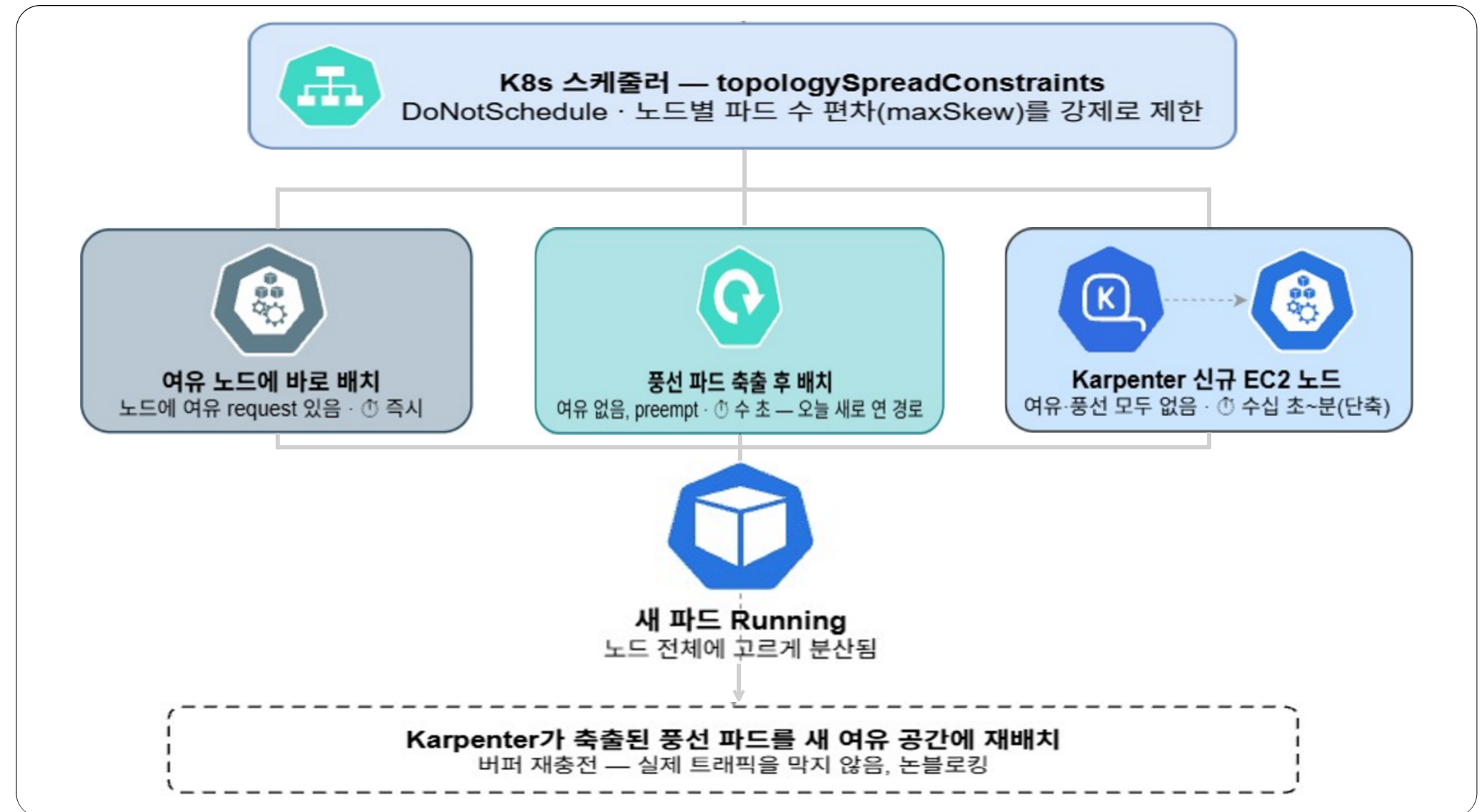
① 파드를 늘리려는 신호



두 층이 함께 움직입니다.

KEDA는 파드를, Karpenter는 노드를 늘립니다.
파드만 늘려도 없을 자리가 없으면 소용이 없습니다.

② 새 파드를 어디에 었을 것인가

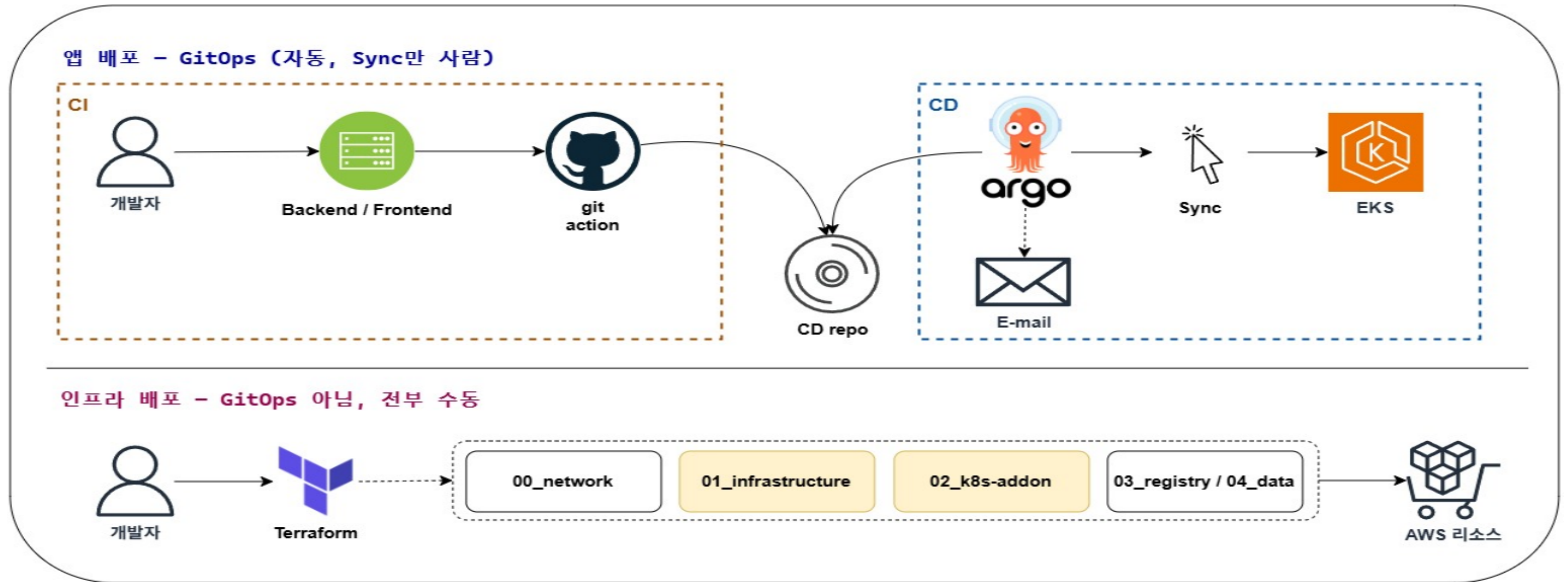


오버프로비저닝

아무 일도 하지 않는 파드로 노드 여유를 미리 잡아둡니다
새 노드를 기다리지 않고 그 자리를 바로 씁니다.

스케일링 기준

KEDA target CPU 70%, cooldownPeriod 300초.
임계치와 대기 시간을 어떻게 잡느냐가 반응 속도를 결정



빌드는 자동, 배포는 수동

Push부터 CD 레포 커밋까지 자동입니다.
OutOfSync를 메일로 알리고 Sync만 팀이 누릅니다.

인프라 삭제·생성 순서의 중요성

상위 리소스나 이를 관리하던 주체가 사라져 하위 리소스가 고아로 남아, 삭제 및 생성에 실패합니다.

비용 관리

01 · 02 레이어만 야간에 삭제해 비용을 줄입니다.
네트워크와 데이터는 유지됩니다.

✓ Build and push backend image ①

```
81 #10 naming to 727646470302.dkr.ecr.ap-northeast-2.amazonaws.com/team5/ecr/qket:backend-2d5fbd9 done
82 #10 DONE 1.0s
83 The push refers to repository [727646470302.dkr.ecr.ap-northeast-2.amazonaws.com/team5/ecr/qket]
84 17de2d388579: Preparing
85 98c92add586d: Preparing
86 c4374c48c6ac: Preparing
87 d924981500f4: Preparing
88 f5360af548c1: Preparing
89 f4c8fa7e3b37: Preparing
90 87c9dd9a3da3: Preparing
91 c0359c5b6ede: Preparing
92 f4c8fa7e3b37: Waiting
93 87c9dd9a3da3: Waiting
94 c0359c5b6ede: Waiting
95 d924981500f4: Layer already exists
96 f5360af548c1: Layer already exists
97 f4c8fa7e3b37: Layer already exists
98 87c9dd9a3da3: Layer already exists
99 c4374c48c6ac: Pushed
100 17de2d388579: Pushed
101 98c92add586d: Pushed
102 c0359c5b6ede: Pushed
103 backend-2d5fbd9: digest: sha256:b39b7cdd932ee13ee7e196f466998b36db62aa8d907aaf9abc6f1032917a5b65 size: 1994
104 Pushed image: 727646470302.dkr.ecr.ap-northeast-2.amazonaws.com/team5/ecr/qket:backend-2d5fbd9
```

✓ Update image tag in qKet/CD ②

```
1 ▶ Run IMAGE="$REGISTRY/$ECR_REPOSITORY:$TAG"
23 [main 6b92275] chore: update backend(prod) image to backend-2d5fbd9
24 1 file changed, 1 insertion(+), 1 deletion(-)
25 To https://github.com/qKet/CD
26 807d005..6b92275 main -> main
```

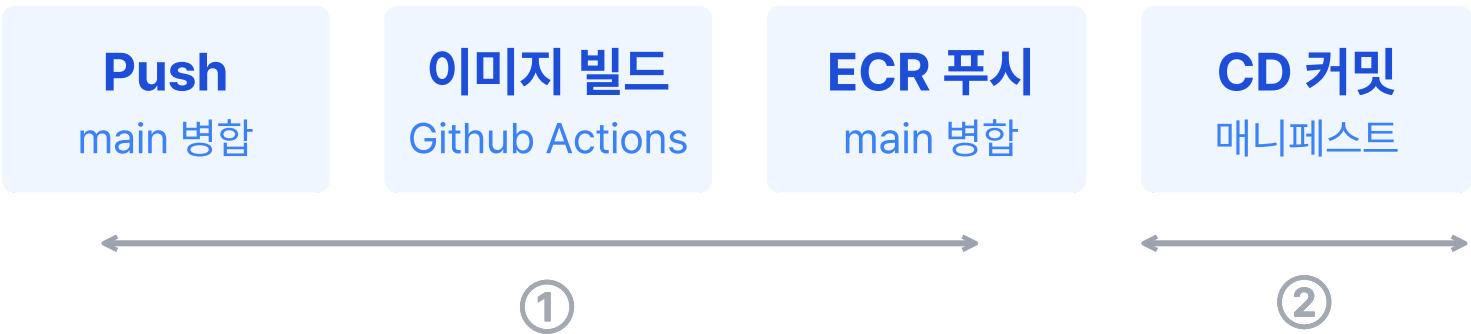
- 빌드 · 푸시

Push 한 번에 빌드 → ECR 업로드까지 자동입니다.
이미지 태그는 커밋 SHA라 어떤 코드인지 추적합니다.
- 자격 증명

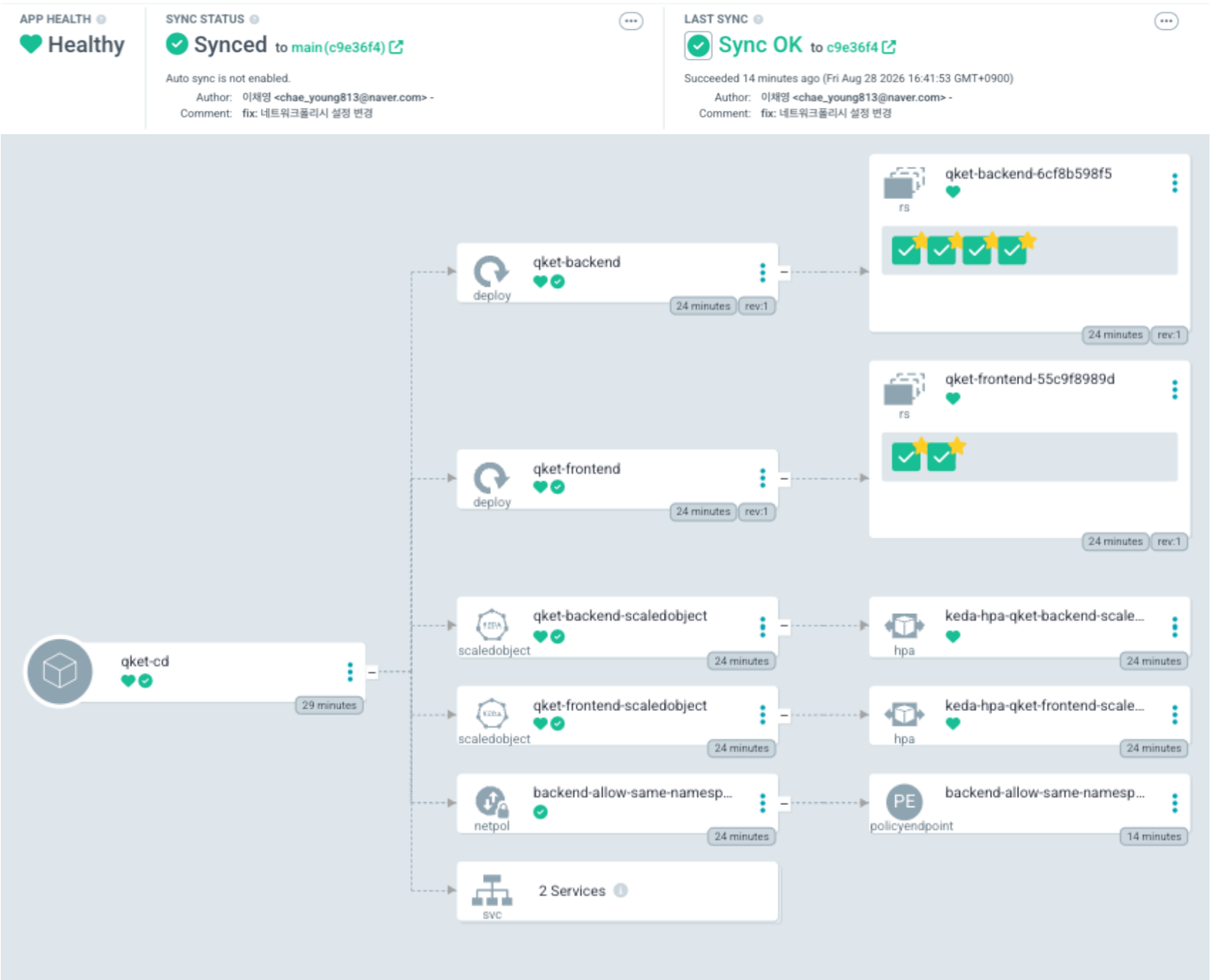
개인 토큰 대신 Github App 토큰을 받습니다.
특정 계정에 의존하지 않고, 장기 키가 저장되지 않습니다.
- CD 연결

CI가 CD 저장소의 매니페스트 태그를 직접 커밋합니다.
사람이 이미지 태그를 옮겨 적는 단계가 없습니다.

CI / CD 흐름



여기까지가 전부 자동입니다.
사람이 개입하는 건 다음 창의 Sync 버튼 하나 뿐 입니다.



qket-cd		qket-cd-release	
Project:	default	Project:	default
Labels:		Labels:	
Status:	♥ Healthy ✓ Synced	Status:	♥ Healthy ✓ Synced
Repository:	https://github.com/qKet/CD.git	Repository:	https://github.com/qKet/CD.git
Target Revision:	main	Target Revision:	main
Path:	helm	Path:	helm
Destination:	in-cluster	Destination:	in-cluster
Namespace:	qket-prod	Namespace:	qket-release
Created At:	08/28/2026 16:28:49 (28 minutes ago)	Created At:	08/28/2026 16:28:49 (29 minutes ago)
Last Sync:	08/28/2026 16:41:53 (15 minutes ago)	Last Sync:	08/28/2026 16:43:08 (14 minutes ago)
<button>↻ SYNC</button> <button>↻ REFRESH</button> <button>✖ DELETE</button>		<button>↻ SYNC</button> <button>↻ REFRESH</button> <button>✖ DELETE</button>	

환경 분리

같은 레포와 Helm 차트를 qket-prod · qket-release 두 네임스페이스로 나눠 배포합니다.

배포 방식

Auto-sync is not enabled - 자동 동기화를 끄고 Sync 버튼을 눌러 배포합니다.

현재 상태

두 환경 모두 Healthy · Synced, 커밋 메시지와 작성자까지 함께 기록됩니다.

Deployment뿐만 아니라 KEDA ScaledObject와 NetworkPolicy까지 ArgoCD가 관리합니다. 스케일링 설정과 통신 정책도 Git에 선언되어 있습니다.

보안 방식

네트워크

Private Subnet · NetworkPolicy · SSM Bastion
SSH 포트를 열지 않고, 관리 도구는 팀 IP 허용목록 뒤에만 노출

권한

IRSA (IAM Roles for Service Accounts)
노드 전체가 아니라 파드 단위로 최소 권한만 부여

자격증명

External Secrets Operator + Secrets Manager · CI - OIDC
Git에도 CI에도 저장된 키가 없습니다. 실행마다 임시 자격증명을 발급합니다

애플리케이션

인터셉터 기반 인가 · 대기열 우회 차단
컨트롤러마다 흩어져 있던 검사를 한 곳으로 모아 강제

관측 방식

Prometheus 지표 · AMP 연동

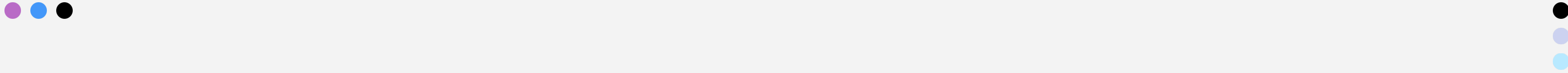
CPU · 메모리 · 파드 수. AMP에 영구 저장해
클러스터를 매일 지워도 이력이 남습니다.

Loki 로그

파드가 계속 바뀌어도 로그가 남습니다.
실패한 요청의 원인을 추적했습니다.

Grafana Faro 프론트엔드

브라우저에서 수집합니다.
서버는 정상인데 사용자 체감은 느린 구간을 볼 수 있습니다.



Chapter 4

부하 테스트 및 성능 개선

- 부하 테스트 설계
- 1차 측정 결과
- 개선 시도와 역효과
- 병목 원인 분석
- 개선 결과와 용량 산정

1,000

스파이크 · 지속 부하
90초 폭증과 25분 지속
2개 시나리오

2,000

전 구간 시나리오 · 실측에 사용
로그인 → 대기열 → 좌석 선택 → 예매
같은 스크립트를 두 규모로 실행

4,000

10,000

최대 규모
대기열 유무 2개 버전
산정 기준으로 활용

측정 대상

로그인
성공률



대기열
대기 시간



좌석
응답 지연



예매
성공 · 포기
동시성

시나리오 수정

처음에는 각 VU가 시간 내내 시나리오를 반복해서,
"500명"이 실제로는 500명이 아홉 번씩 재시도한 부하였습니다.
오픈런은 그 순간 한 번씩 올리는 것이므로 VU 수가
곧 사람 수가 되도록 변경했습니다.

- 서버 측은 노드 CPU · Pod 재시작 · 스로틀링 · HikariCP 커넥션을 Grafana에서 동시에 관찰했습니다.

테스트 규모

2,000

8/21 16:20 - 전구간

backend replica

2 → 4

목표 8개에 도달 못함

Pod 재시작

4 회 x 4개

기동 중 반복 사망

노드 CPU

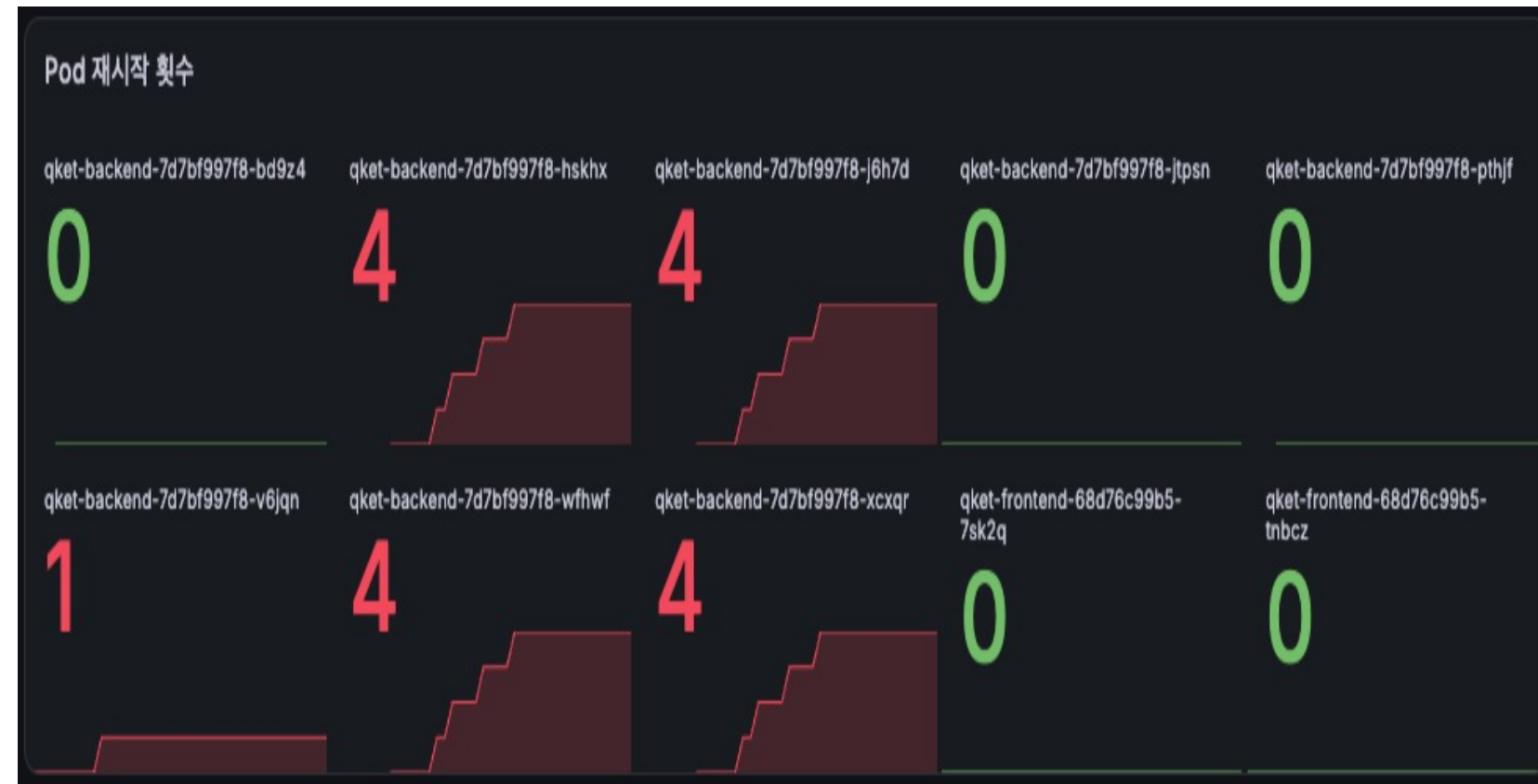
100%

한 노드만 반복

노드 CPU 사용률



Pod 재시작 횟수



관찰

오토스케일링은 동작했지만 새 파드가 이미 CPU가 꽉 찬 노드에 배치되어, 기동 중 파드가 자원을 못 받고 죽기를 반복했습니다.

가설

파드가 한 노드에 몰리는 부분이 문제다.
노드별로 고르게 분산시키면 해결될 것이다.

2차 · 8/21 17:05

ScheduleAnyway · 소프트 제약

Karpenter가 새 노드를 하나만 만들어 파드 4개가 그대로 몰림

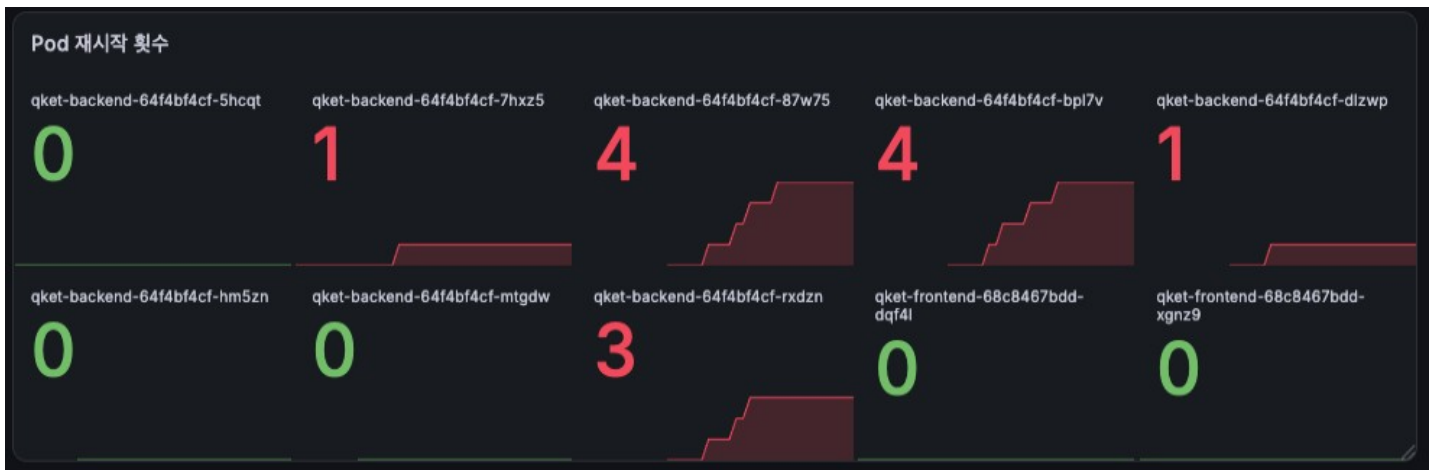
3차 · 8/21 18:25

DoNotSchedule · 하드 제약

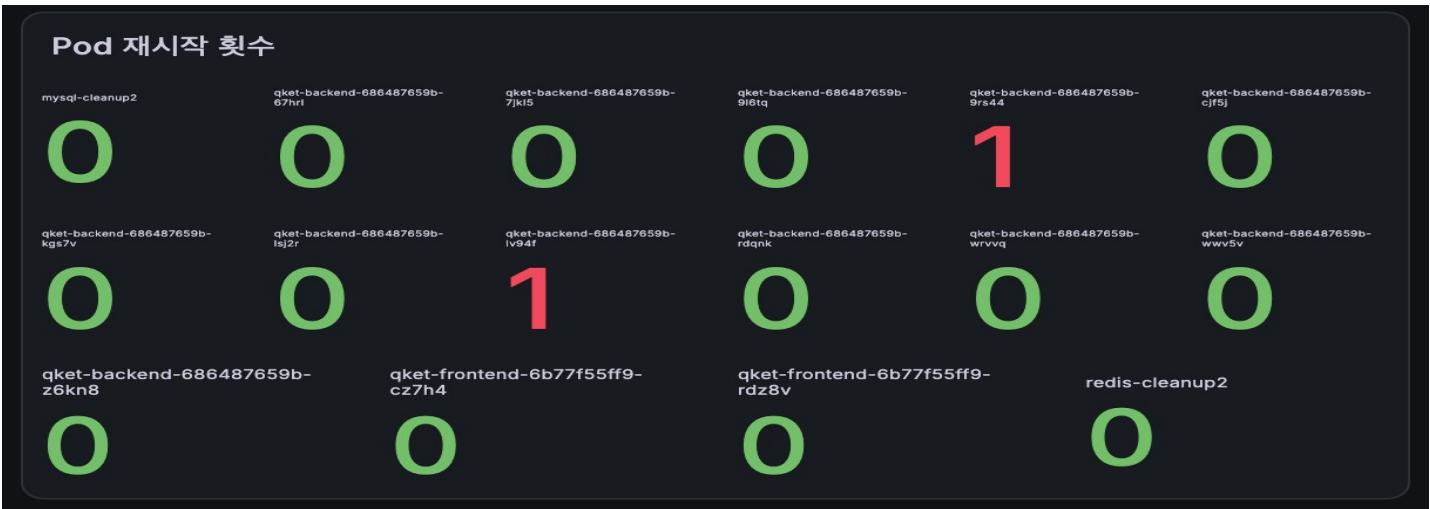
분산은 강제됐지만 새 노드를 기다리느라 스케일업이 느려짐

목표 달성 - 재시작

2차 - 재시작



3차 - 재시작



예상 못한 결과

로그인 성공률

53% 100%

HPA CPU

399% 70%

요청 실패율

3.43%

대기열 진입 성공률

87%

예상 밖 실패

129건 · 임계 초과

가장 오래 걸린 5%

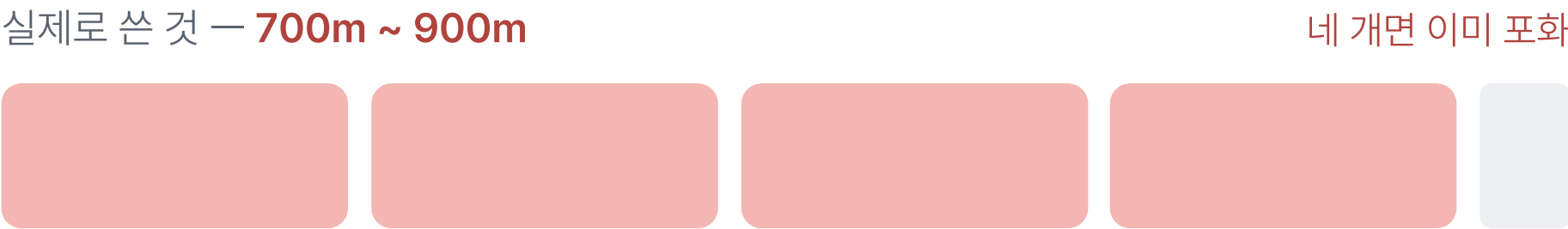
9분 47초

스케일업을 기다리는 동안 이미 떠 있던 파드 두 개가 2,000명의 로그인을 그대로 받았습니다.

같은 노드, 세 개의 계산



파드 4개를 엮고도 자리가 남는다고 계산



t3.xlarge · 4,000m · 실측



노드당 수용 파드 수가 절반으로 줄어 분산이 자연스럽게 강제됨

함께 확인한 것

로그인은 CPU를 많이 씁니다.

BCrypt 검증이라 요청당 CPU 비용이 큼니다.
KEDA가 반응하기 전 파드 2개가 2,000명을 그대로 받았습니다.

파드 수	응답 시간	로그인 성공률
2	29초	84%
몇 분간 유지	CloudWatch	8/24 재검증

세 가지를 함께 고쳤습니다.

CPU request	250m → 500m · limit 1 → 2
minReplicas	2 → 4 · 스케일 전 기본 흡수력
오버프로비저닝	풍선 파드로 노드 여유를 미리 점유

대기열 대기 · 중앙값

641초

8.2초

78배 단축

접속부터 예매완료까지

11분 6초

45초

15배 단축

2,000 · 4,000명 공통

실패 0건

로그인 · 대기열 진입 100% · Pod 재시작 0

인원을 두 배로 올렸을 때

지표	2,000명	4,000명
요청 실패율	0%	0%
사용한 파드	8	8
초당 처리량	370	582 req/s
대기 시간 중앙값	8.2초	45.1초

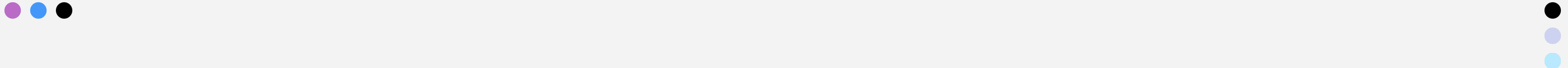
인원이 두 배가 됐을 때 시스템은 실패가 아닌 대기 시간으로 반응했습니다.

규모별 예상

동시 접속	실패율	대기 중앙값	필요 RDS
2,000	0%	8초	t3.small
4,000	0%	45초	t3.small
6,000	—	2~3분	t3.medium
10,000	—	5분 이상	t3.medium 이상

현재 구성

Karpenter 자동 조절 · RDS db.t3.small · Redis cache.t3.micro
dbPoolSize 12 x Pod 8 = 96 커넥션 사용 · CPU request 500m / limit 2



Chapter 5

트러블슈팅 및 피드백

- 장애 사례 - 1
- 조치와 재발 방지
- 장애 사례 - 2
- 멘토 피드백 및 개선 내역

장애 사례 ① - 관리자 API 인가 무력화

2026-08-21 · 심각도 높음

증상

로그인 없이 관리자 API 전체에 접근 가능

영향 범위

6 개 컨트롤러 전수

카테고리 · 사용자 · 프로그램 · 메뉴 · 예약내역 · 공연관리

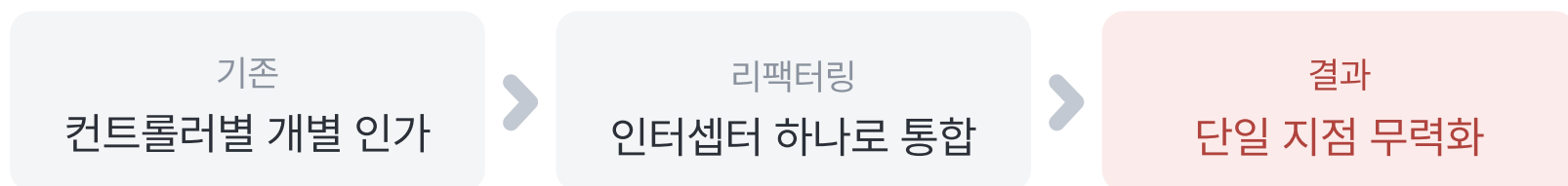
확인 방법

세션 쿠키 없이

GET /api/admin/categories

호출 시 200 OK + 실제 데이터

왜 검사가 통째로 건너뛰어졌나



인가 검사를 인터셉터 하나에 모았는데, 그 하나가 뚫렸을 때를 대비한 2차 검증이 없었습니다.

왜 테스트에서 안 잡혔나

단위 테스트

Context-Path 없이 실행되서
경로가 /admin/ ... 으로 나옴 → 통과

실제 배포

Context-Path /api 가 붙어
경로가 달라짐 → 무력화

원인은 메서드 한 줄

문제 코드

```
String uri = request.getRequestURI();  
// → "/api/admin/categories"  
// startsWith("/admin") 은 항상 false
```

수정 코드

```
String path = request.getServletPath();  
// → "/admin/categories"
```

원인 규명

경로 비교가 항상 false임을 확인

코드 수정

getServletPath() 전면 교체

긴급 배포

Hotfix PR · release 브랜치 머지

전수 검증

6개 API 모두 401 / 403 차단 확인

같은 일이 다시 없도록

01

인증 없는 호출 테스트 추가

이번 버그는 실제 서버에 올렸을 때만 나타났습니다.
포함된 주소로 로그인 없이 관리자 API를 호출하는
테스트를 만들어, 배포할 때마다 자동 실행합니다.

02

권한 검사 이중화

지금은 한 곳에서만 권한을 검사합니다.
각 기능에도 검사를 따로 걸어 두 번 막습니다.

03

인증 코드 리뷰 절차 강화

로그인·권한 관련 코드는 한 사람 승인으로
병합하지 않습니다. 문제가 된 경로 조회 방식도
가이드로 남깁니다.

장애 사례 ② - 부트스트랩 노드 과부하

2026-08-19 · 심각도 높음

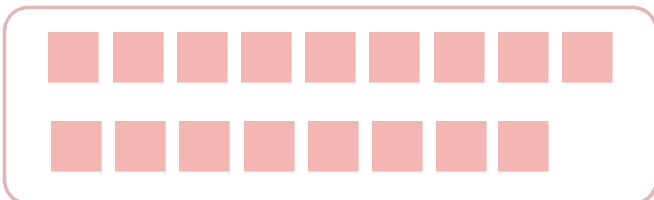
부트스트랩 노드 1대에

34 개 파드

고정 노드 CPU

92%

개선 전 · 1대



개선 후 · 2대



어떻게 발견했나

01

2,000명 부하 테스트 중
고정 노드 CPU가 92% 급증

02

동적 노드는 여유 있는데
고정 노드만 포화

03

인프라 파드 34개가
한 대에 몰려 있었음

- Karpenter가 만드는 동적 노드만 보고 있었고, 스케일링에서 제외된 고정 노드는 관측 밖이었습니다.

근본 원인 - 요청량이 비어 있었습니다.

문제 설정

QoS · BestEffort

```
resource: {}
```

요청량이 없으면 노드가 부족할 때

커널이 가장 먼저 축출

수정 설정

QoS · Burstable

```
resources:
```

```
requests:
```

```
cpu: 100m
```

```
memory: 128Mi
```

조치

QoS 등급

BestEffort → Burstable

노드 구성

1대 → 2대 · 단일점 제거

관측 범위

고정 노드도 대시보드에 포함

반영한 것 4건

받은 피드백

코드와 배포 설정을 분리하라

코드 · YAML 레포를 분리하고, CI는 이미지 빌드까지만 담당, 이후 배포는 ArgoCD가 수행

장키 키 종속 문제 해결

CI는 AWS 키를 저장하지 않고 실행마다 OIDC 임시 인증만 사용, 앱 시크릿은 Secrets Manager에서 자동 동기화

Ingress 대신 Gateway API 검토

nginx ingress controller 가 운영을 중지, 또한 ingress 문법이 이노테이션 의존적이라 통일성이 없어 유지보수가 어려워 gateway api를 사용하기로 결정

장애 시 로그를 어떻게 찾는가

Loki를 도입해 Grafana에서 전체 로그를 한 번에 조회하도록 구성

반영하지 않은 것 1건

받은 피드백

prod 환경을 별도 클러스터로 분리하지 않는 이유는

클러스터 분리 시

비용 2배

선택한 방식

네임스페이스

클러스터 하나를 네임스페이스로 나눠 release · prod를 분리하고, NetworkPolicy로 통신을 격리했습니다.
학습 프로젝트의 예산 안에서 격리 수준과 비용을 맞바꾼 판단입니다.



Chapter 6

시연 및 자체 평가

- 시연 영상
- 완성도 평가
- 자체 평가와 추후 개선점
- 자체 평가 의견



이 곳에는 표와 관련된 서브내용을 입력해보세요.

서비스 기능

핵심 예매 플로우 완료

회원가입 → 대기열 → 좌석 선택 → 결제 → 마이페이지
전 구간 동작

관리자 기능 완료

공연 · 회차 · 좌석, 카테고리 · 메뉴 · 프로그램,
사용자 권한, 예약 이력

결제 · 환불 완료

Toss 결제 승인 · 취소 · 환불, 결제 내역 조회

회원 · 인증 완료

이메일 인증 회원가입, 소셜 로그인 3종, 비밀번호 찾기

PR 리뷰 · 컨벤션 완료

모든 변경은 Pull Request로만 병합하고
브랜치 전략과 커밋 컨벤션을 통일했습니다.

감상평 · AI 판별 완료

예매한 회차만 작성 가능, OpenAI로 스포일러 자동 판별

품질

코드 품질 · 유지보수성 완료

CSS 값을 `:root` 변수로 표준화, 반복 UI를 재사용
컴포넌트로 전환

문서화 완료

위키와 docs 문서가 실시간으로 갱신되며
문서화 역할을 담당

알림 완료

예매 확정 · 취소 메일, 공연 오픈 알림 신청

테스트 커버리지 미완료

Backend는 컨텍스트 로드 테스트 1건
Frontend는 테스트 파일 없음.
k6 부하 테스트로 일부만 보완

인프라 · 운영

인프라 코드화 완료

Terraform 모듈을 기능 단위로 나누고 5개 독립 레이어로
격리해, 변경 범위를 좁히고 재사용할 수 있게 구성했습니다.

CI / CD 신뢰성 완료

빌드 후 CD 레포의 매니페스트 태그를 자동 갱신하고,
ArgoCD OutOfSync 알림을 받아 수동 승인으로 배포합니다.

보안 · 시크릿 자동화 완료

GitHub Actions 인증을 OIDC로 전환해 장기 자격증명을 없애고,
Secrets Manager와 ESO를 연계해 시크릿 주입을 자동화했습니다.

대기열 실측 부하 테스트 완료

좌석 2,000석에 4,000명이 동시 접속하는
오픈런을 재현. 중복 예매 0건으로 목표 달성

관측성 부분 완료

Prometheus · Grafana · Loki · Faro · AMP
ArgoCD 매일 알림까지 동작. 외부 헬스체크와
Alertmanager 알림은 미구현

잘한 점

부하 테스트

10 회 이상

추측하지 않고 측정했습니다

분산 정책으로 풀려나 로그인이 53%로 떨어졌고, 사용자 지표를 함께 봤기에 되돌릴 수 있었습니다.

클러스터 재생성

매일 자동

전 구간을 코드로 관리했습니다

Terraform과 GitOps로 매일 밤 지우고 아침에 다시 만드는 구조를 갖췄습니다.

4,000명 동시 예매

중복 0 건

실패를 가정하고 설계했습니다

분산 락이 풀려도 조건부 갱신이 막도록 이중으로 구성했습니다.

의사결정 · 장애

그때마다 기록

결정과 실패를 남겼습니다

왜 골랐는지, 무엇이 왜 실패했는지를 위키에 남겨 같은 실수를 반복하지 않았습니다.

아쉬운 점

지금 아쉬운 것

테스트 코드가 거의 없습니다

Backend 1개 · Frontend 0개

알림이 없습니다

대시보드를 열어봐야 문제를 알 수 있음

목표 규모를 다 검증하지 못했습니다

4,000명까지 실측 · 이후는 시뮬레이터로 산정

다음에 할 것



테스트 코드 작성 · PR 빌드 검증

Backend · Frontend 테스트 작성 후 PR 빌드체크에 실행 단계 추가



자동 알림과 외부 헬스체크 구축

Alertmanager 임계치 알림 · Blackbox Exporter 외부 감시 · ALB 에러율 연동



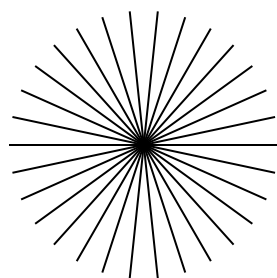
10,000명 규모 최종 재검증

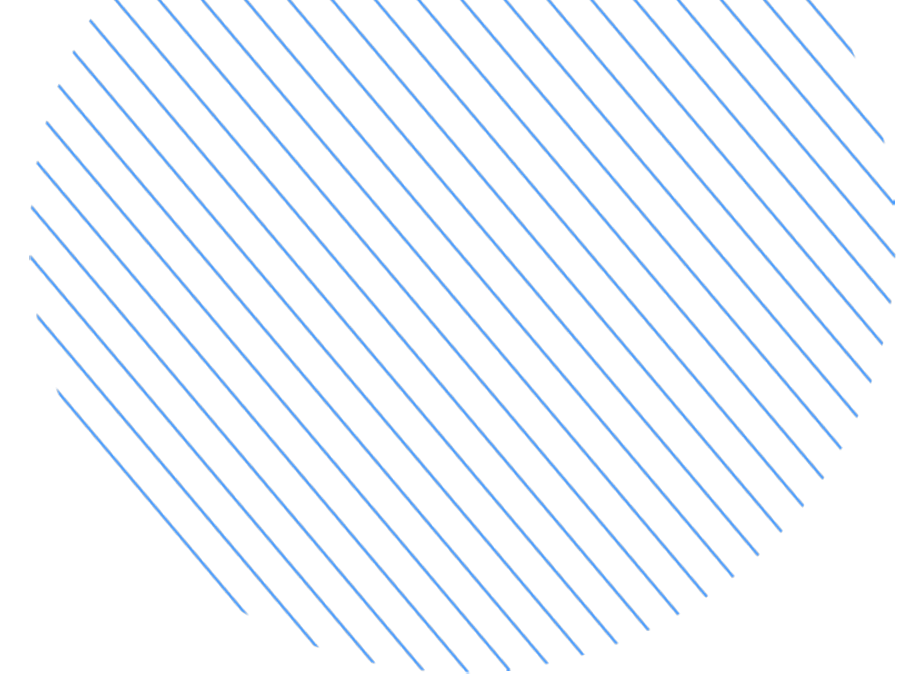
RDS 등급 상향과 ALB 사전 예열 · KEDA 스케일업 반응 개선



Q&A

질의응답 시간입니다





for your time



Thank you

